

あたらしいを、あたりまえに

J:COM

開発エンジニアが取り組む DevSecOps

～ GitHub Enterprise × Azure での実践～

2026/2/19 OctoNihon Form #4

JCOM株式会社

次期フロントシステム開発部 渡辺 一宏





渡辺一宏

JCOM株式会社 次期フロントシステム開発部

Webアプリケーションの開発
DevOps環境の構築・運用
クラウドインフラの構築・運用



Microsoft Azure, Developer Technologies
(Azure Infrastructure as Code, Web Development)

@kaz_29

何をしている部署か？

あたらしいを、あたりまえに

J:COM

J:COM Online Shop | お申し込み | 料金シミュレーション | カートを見る | ご利用ガイド | よくあるご質問 | J:COMトップ

J:COMのサービスを検討する

検討中のサービスがある方



サービスから選ぶ



自分に合うサービスがわからない…



おてがるサービス診断



サービスから選ぶ

☐ ネット

くわしく

☐ テレビ

くわしく

☐ 固定電話

くわしく

☐ 電気

くわしく

スマホ

くわしく

スマホ お申し込み



スマホ 料金シミュレーション



J:COM マイページ

トップページ | ご利用金額 | ご契約情報 | お手続き一覧 | サービスの追加・変更 | トラブル診断

ログインするとご利用状況の確認やお手続きが行えます。

ログイン



ID、パスワードをお忘れの方はこちら
らーばんねっとサービスご利用中のお客さまは、こちらのページからご契約内容の確認、お手続きができます。

マイページの機能をご利用になりたい方はJ:COMパーソナルIDを登録してください

J:COMパーソナルIDの新規登録



請求金額・ご契約内容の確認、各種サービスの追加・変更などがサイト上で簡単に行えます

・ログ
・J:CO
・メー
・各種



MY J:COM ¹²⁺

楽しく・便利な機能が一つに

JCOM Co., Ltd.

iPad対応

「エンターテインメント」内47位

★★★★★ 4.0 • 1.7万件の評価

無料

こちらで表示: Mac App Store

スクリーンショット iPad iPhone



DevOps構成概要

DevSecOpsの前に

DevSecOpsの取組み

- コードの健全性チェック (SonarQube)
- パッケージ更新を自動化 (Dependabot)
- コンテナ脆弱性を一元管理 (Trivy + Advanced Security)

取組み中のMCP活用について

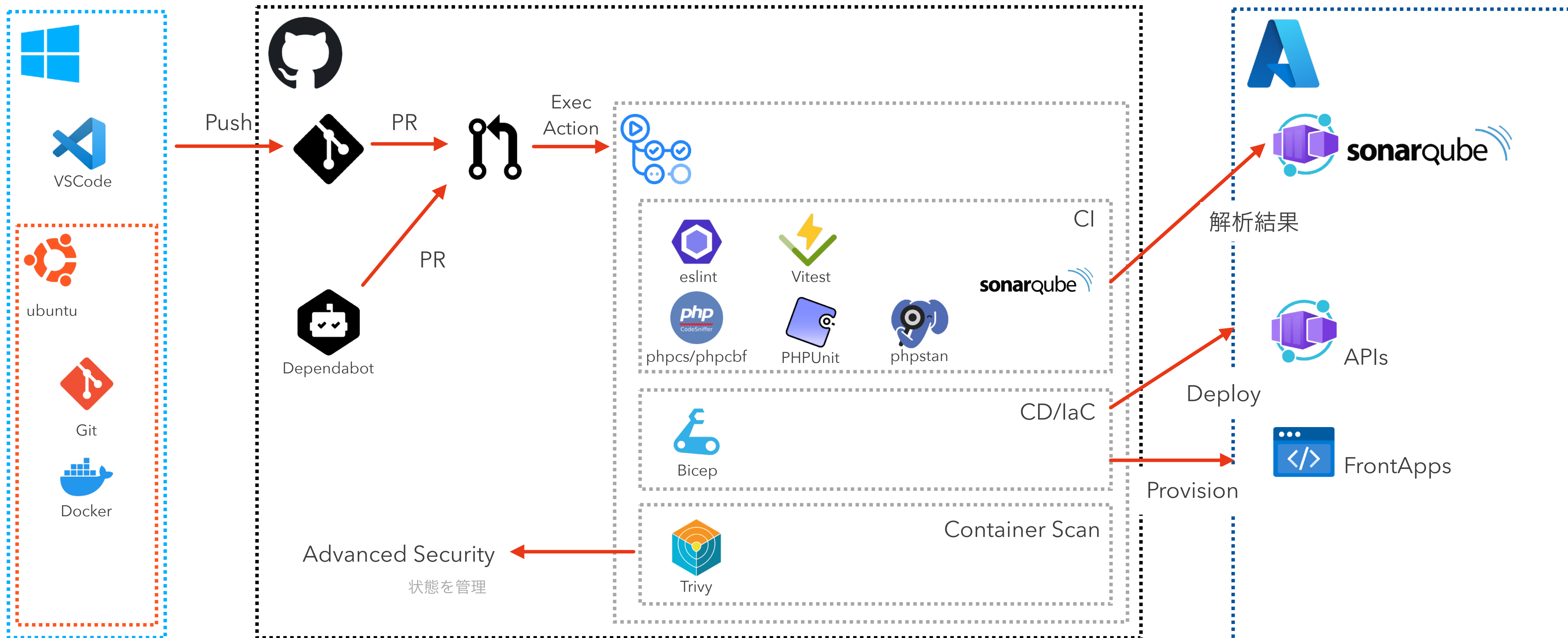
まとめ

DevOps構成概要

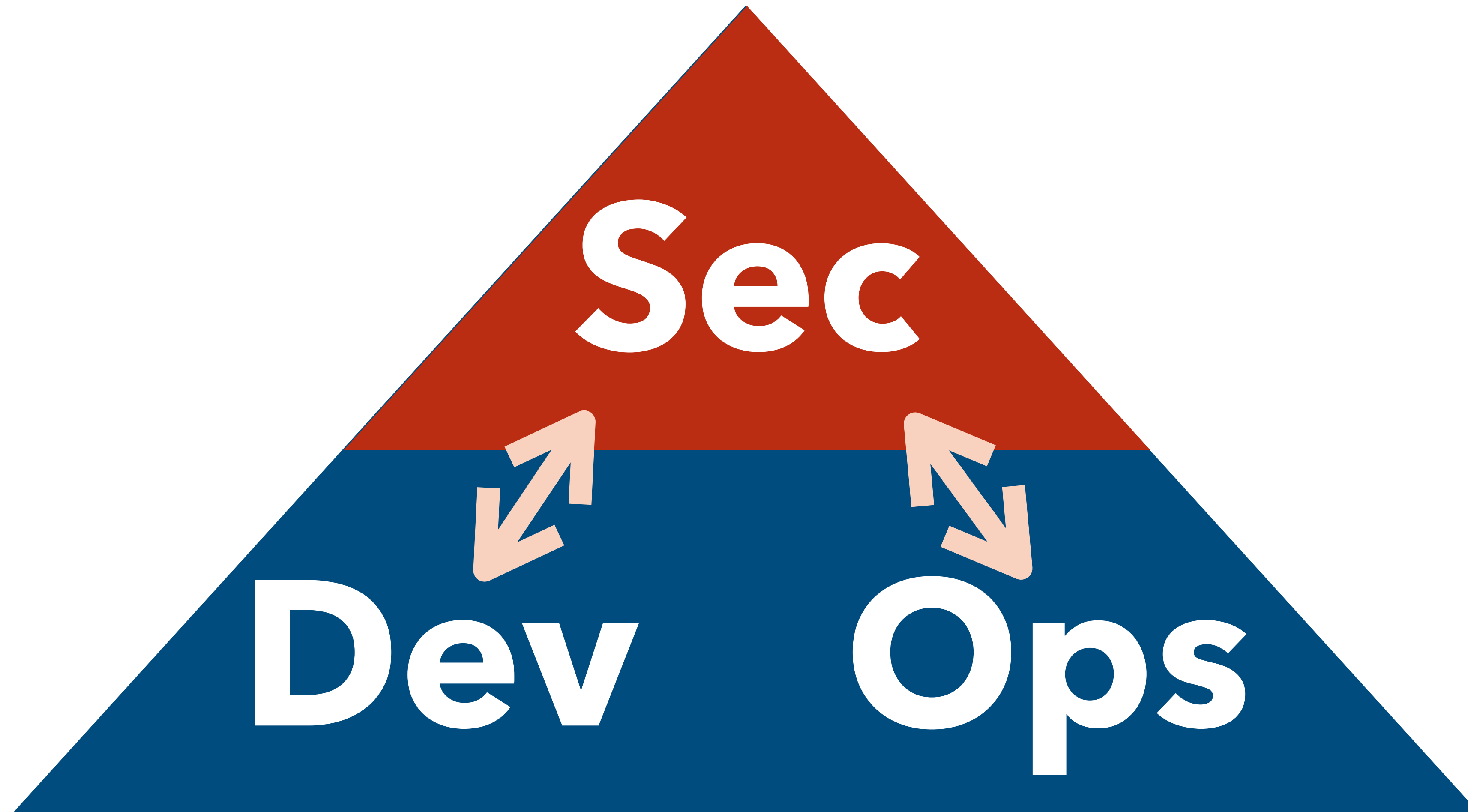
DevOpsの構成概要

あたらしいを、あたりまえに

J:COM



DevSecOps



しっかりしたDevOpsの実現がDevSecOpsの前提

- インフラ生成・設定の自動化(IaC)
 - コードに落とすことで、インフラをソフトウェア開発と同じ手法で管理できる
- バージョン管理
 - 全てのリソース(ソースコード、IaCのコード、DBのマイグレーションなど)
- テストの自動化
 - UnitTest, Functional Test, (E2E Test)
- ビルド・デプロイの自動化
 - ビルドプロセスを全て自動化、ワンボタンで誰でも実行できるように、実行結果を全員で共有
- リアルタイムに情報共有
 - 全員が参加しているチャットで全てを共有、システムに起きていることを通知、コミュニケーションコストの削減

DevOpsの構成概要

CI - PR作成時にUnitTestなどを自動実行

あたらしいを、あたりまえに

J:COM

コンテナのビルド

コーディングスタイルチェック

phpstanによる静的解析

コードの信頼性・保守性向上の施策

- ・ 検知内容の例
 - ・ 型の不一致の検知
 - ・ 存在しないクラスやメソッドの呼び出し
 - ・ 配列キーやプロパティの未定義アクセス
 - ・ 到達不能なコード

UnitTest・Functional Testの実行

SonarQubeによる静的解析

SAST（静的アプリケーションセキュリティテスト）

- ・ 検知内容
 - ・ バグやセキュリティの脆弱性
 - ・ コードの重複、複雑性、コーディング規約違反
 - ・ テストカバレッジの測定
 - ・ 技術的負債の定量化

実行結果をTeamsに通知

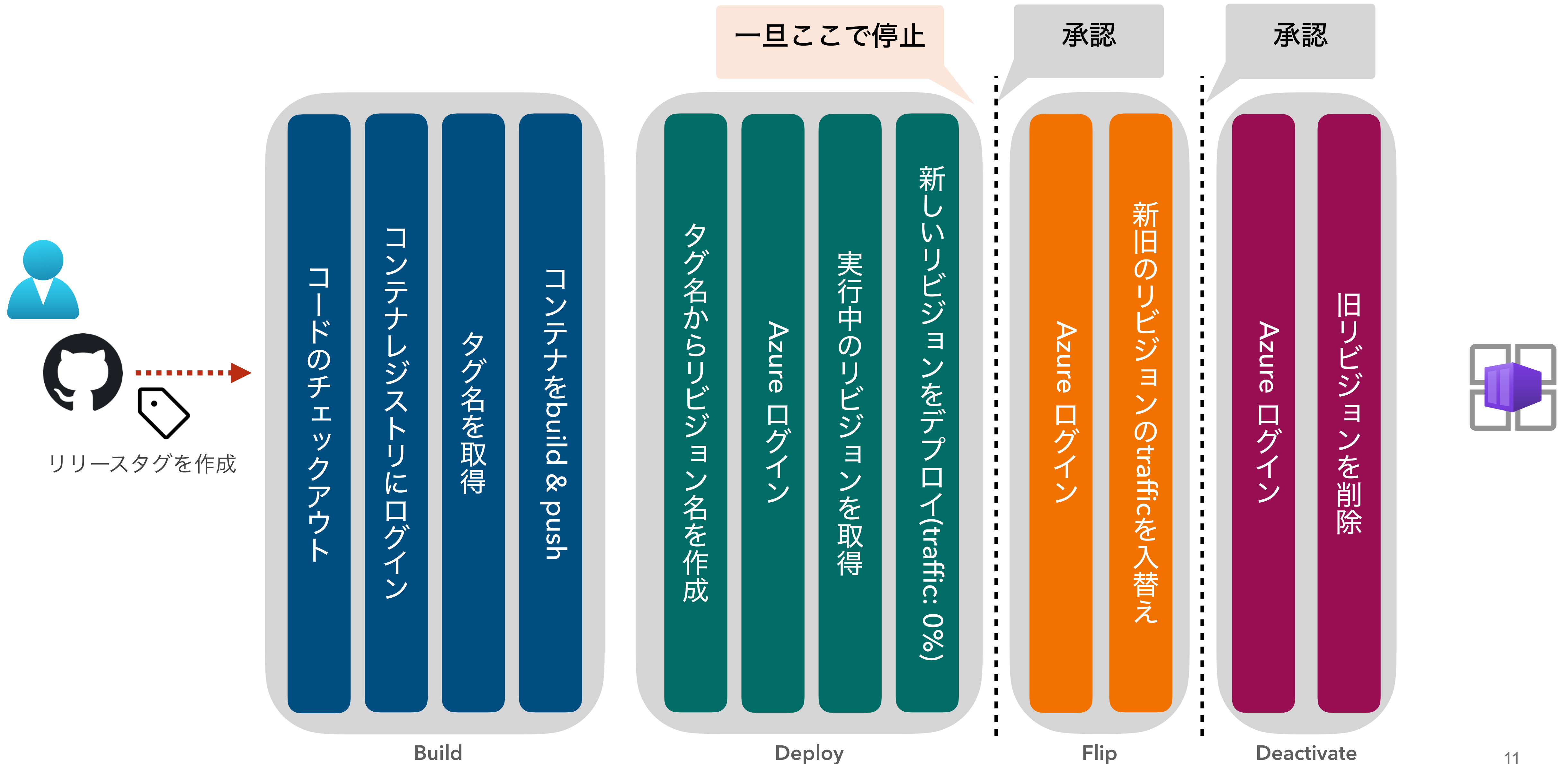
SonaeQube BadgeやPRコメントに投稿

DevOpsの構成概要

デプロイの自動化

あたらしいを、あたりまえに

J:COM

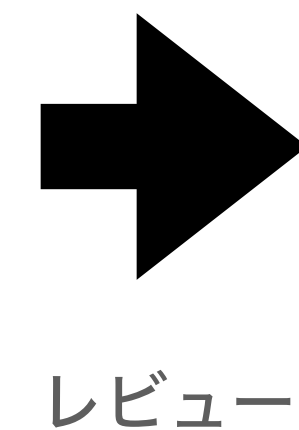
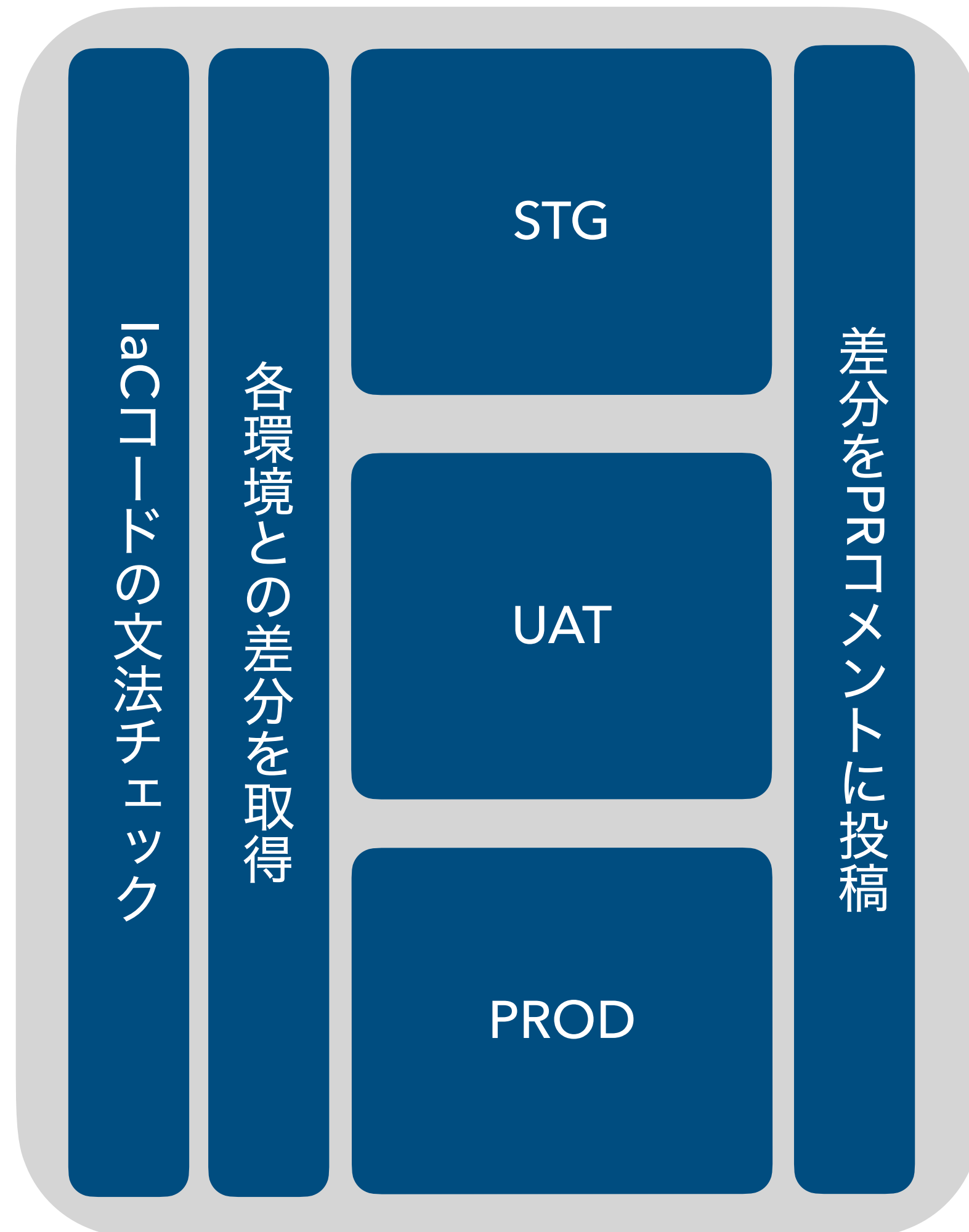
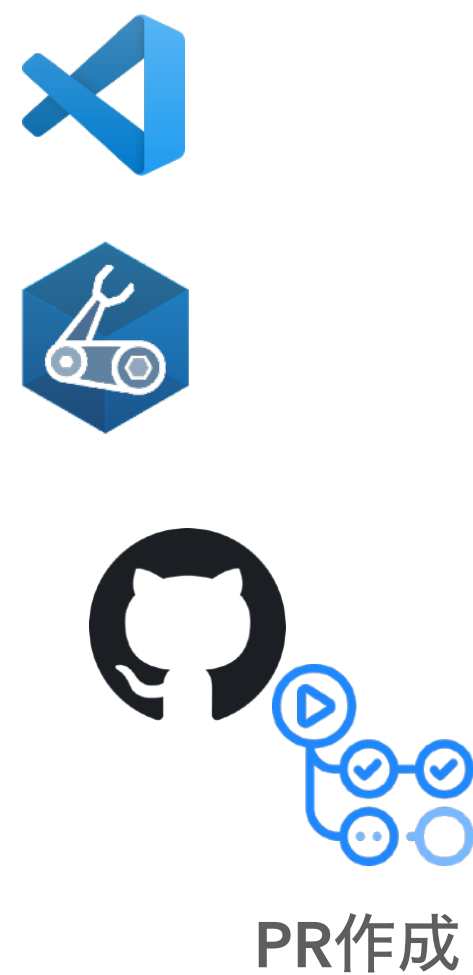


DevOpsの構成概要

IaC - 各環境のプロビジョニングを自動化

あたらしいを、あたりまえに

J:COM

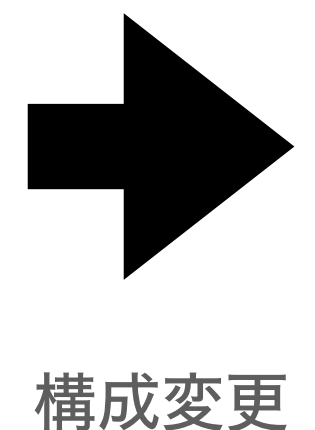


Bicepコードと
差分を比較
問題がなければ
マージ



一旦ここで停止

承認

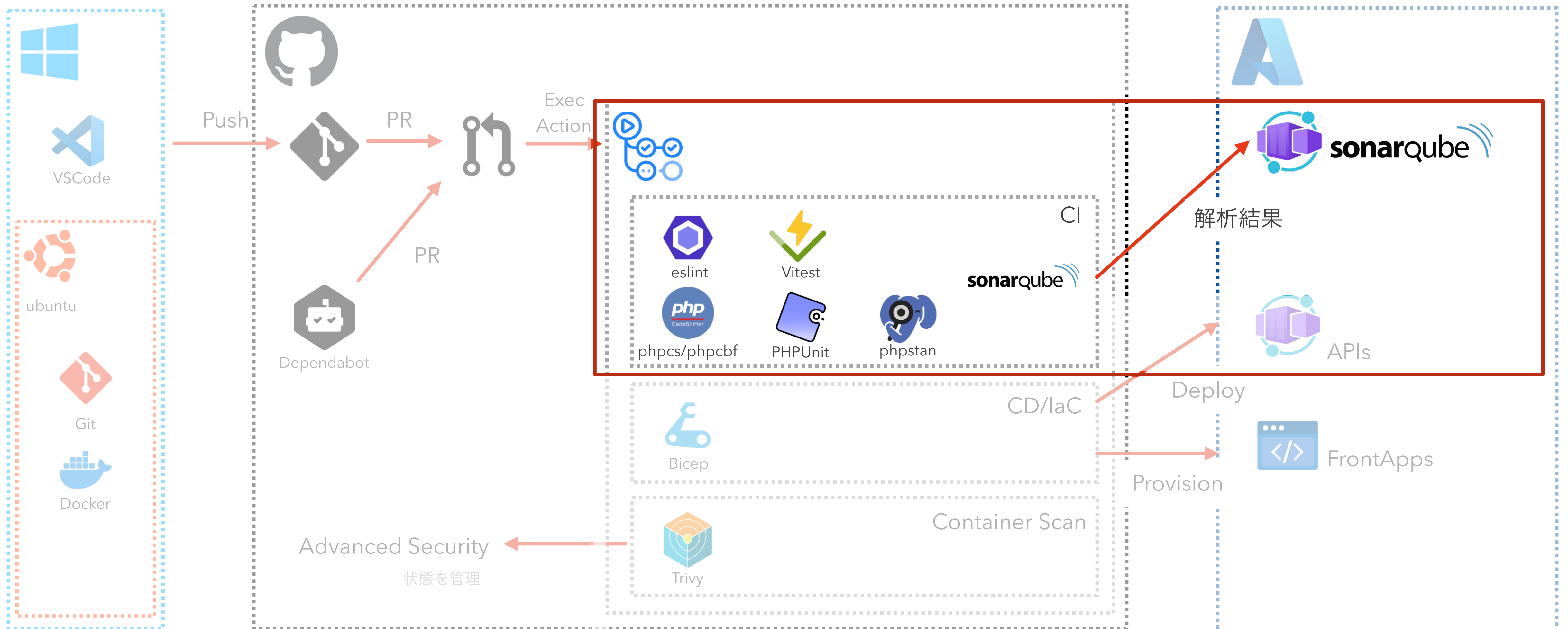


SonarQube

DevOpsの構成概要(再掲)

あたらしいを、あたりまえに

J:COM



sonarqube

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministrationMore

?

W

☆

service /

main

✓

▼

?

OverviewIssuesSecurity HotspotsMeasuresCodeActivity

Project Settings▼Project Information

main

5.8k Lines of Code • Version not provided • Set as homepage

⚠

The last analysis has warnings. [See details](#)

Quality Gate Status ?

✓

Passed



Enjoy your sparkling clean code!

Measures

Last analysis 12 days ago

New Code

Overall Code

Security

0 Open issues

0 H

0 M

0 L

A

Reliability

0 Open issues

0 H

0 M

0 L

A

Maintainability

1 Open issues

0 H

1 M

0 L

A

Accepted issues

64

Valid issues, but not fixed. They represent accepted technical debt.



Coverage

91.4%

On 1.6k New Lines to cover.



Duplications

4.0%

On 5.8k New Lines.



Security Hotspots

0

A

SAST

Static Application Security Testing : 静的アプリケーションセキュリティテスト

Azure上に構築

Community Editionを自前で運用

Azure Container Apps - ビジネスタイム以外は0スケールでコスト0)

Azure Database for PostgreSQL

Azure Storage(Files)

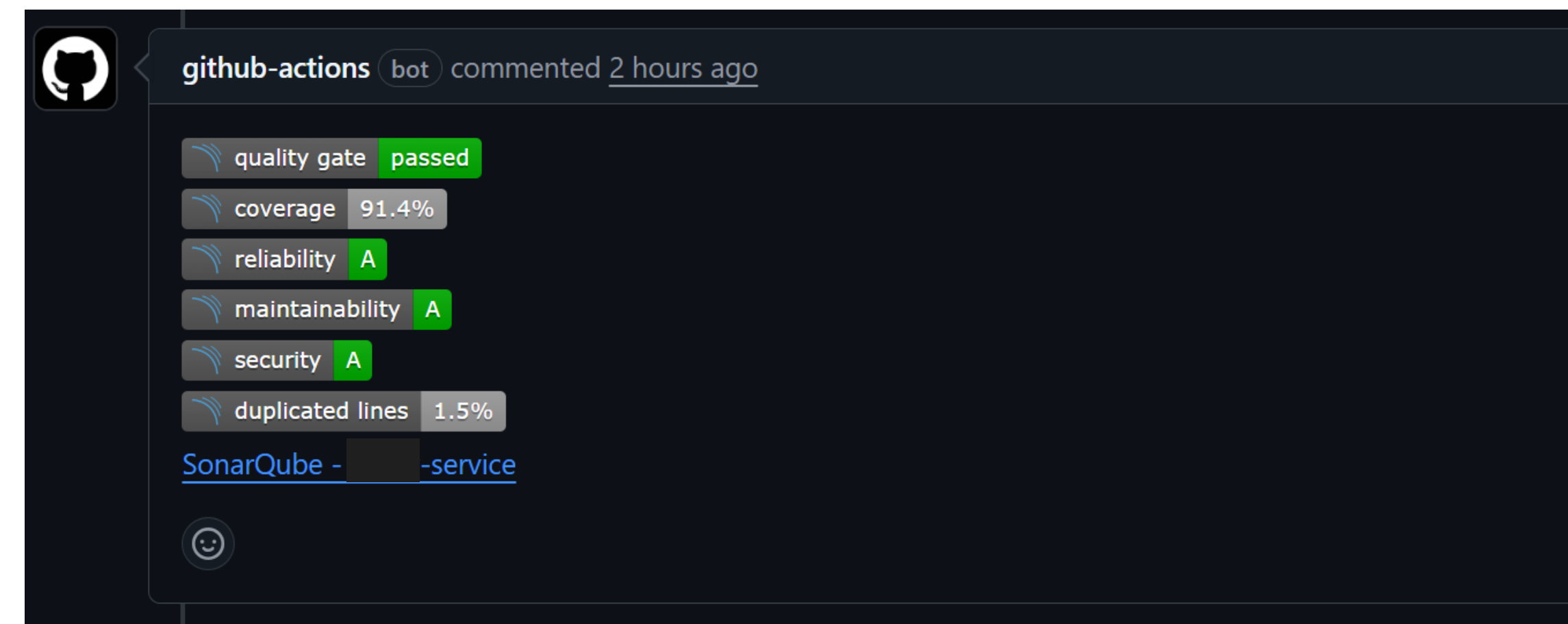
PHP/Typescriptなど全てのリポジトリで利用

Quality Gate

- カバレッジ80%以上
- 現状他はほぼデフォルト、鋭意調整を進めている

運用上のちょっとした工夫

- PRコメントにBadgeを自動投稿
- スプリント終盤に定期チェック・対応



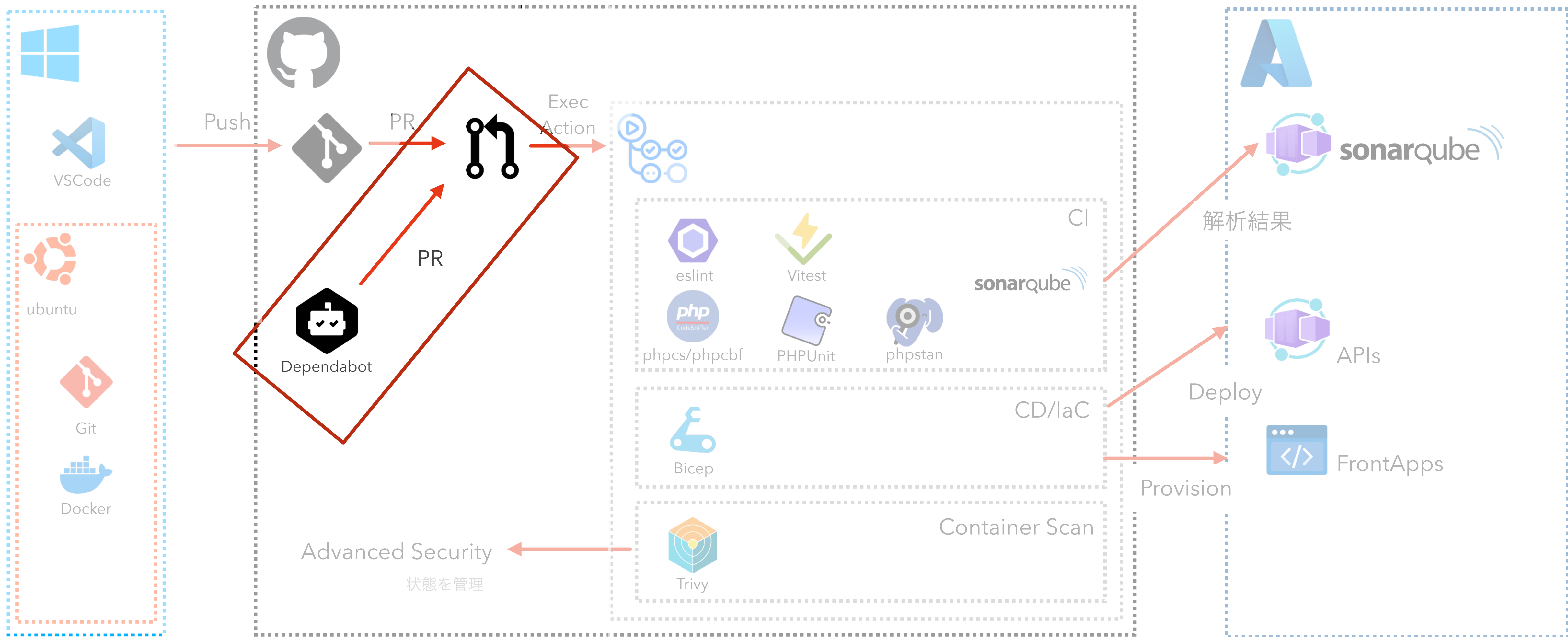
Dependabot



DevOpsの構成概要(再掲)

あたらしいを、あたりまえに

J:COM



GitHub謹製のライブラリ・パッケージの依存関係自動管理ツール

特徴

- バージョンアップを検知
- 脆弱性を検知
 - => 対応するPRを自動生成(互換性のレベルやChangeLogをコメント内に記載)
- CIと組み合わせることでアップデートの手間を軽減



設定

PHP(composer) / TS(npm)両方利用

週次でチェックを実行(月曜朝)

Security Update / Version Update両方有効

Grouped update(特にTSでは必須)

運用

auto-mergeは未使用。compatibilityを見て手動判断

自動テストが通れば、カジュアルにマージ

PRが溜まる問題

スプリント毎に担当立てるなどチーム毎に工夫している



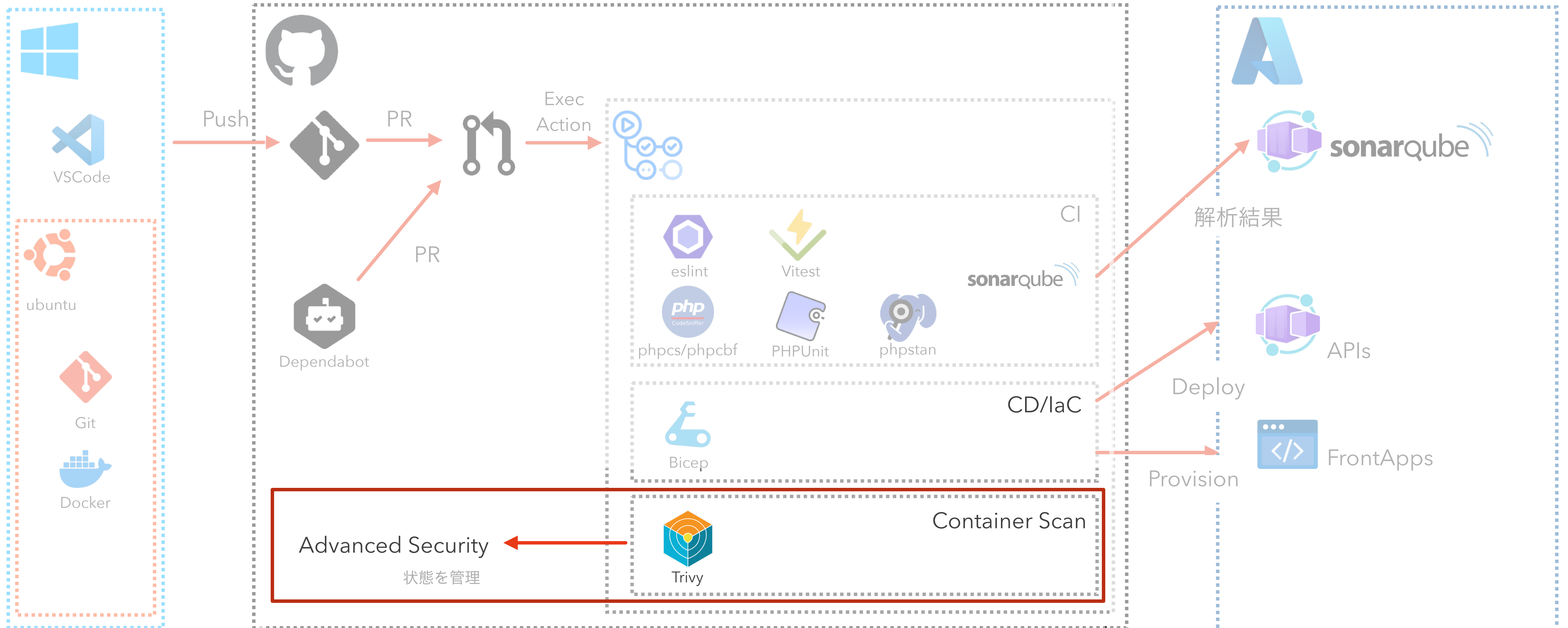
Trivy + Advanced Security



DevOpsの構成概要(再掲)

あたらしいを、あたりまえに

J:COM



OSSの脆弱性スキャナー

対応範囲が広い

コンテナイメージ

ファイルシステム

軽量・高速・CI組み込みが簡単

SARIF形式で出力可能

→ GitHub Advanced Security と連携

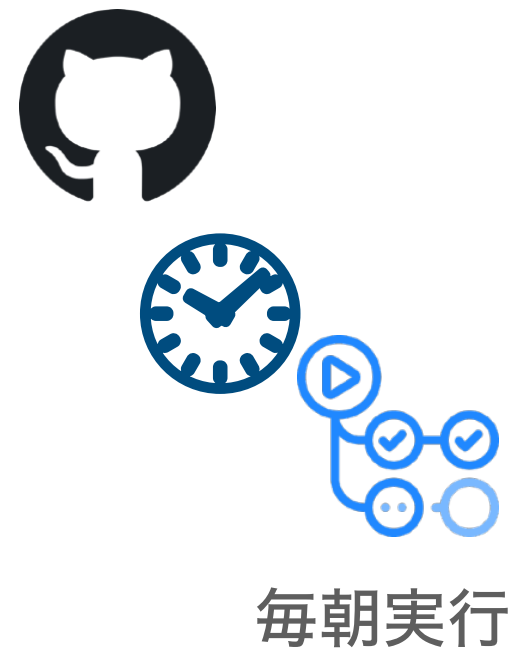


Trivy + Advanced Security

コンテナ脆弱性の一元管理

あたらしいを、あたりまえに

J:COM

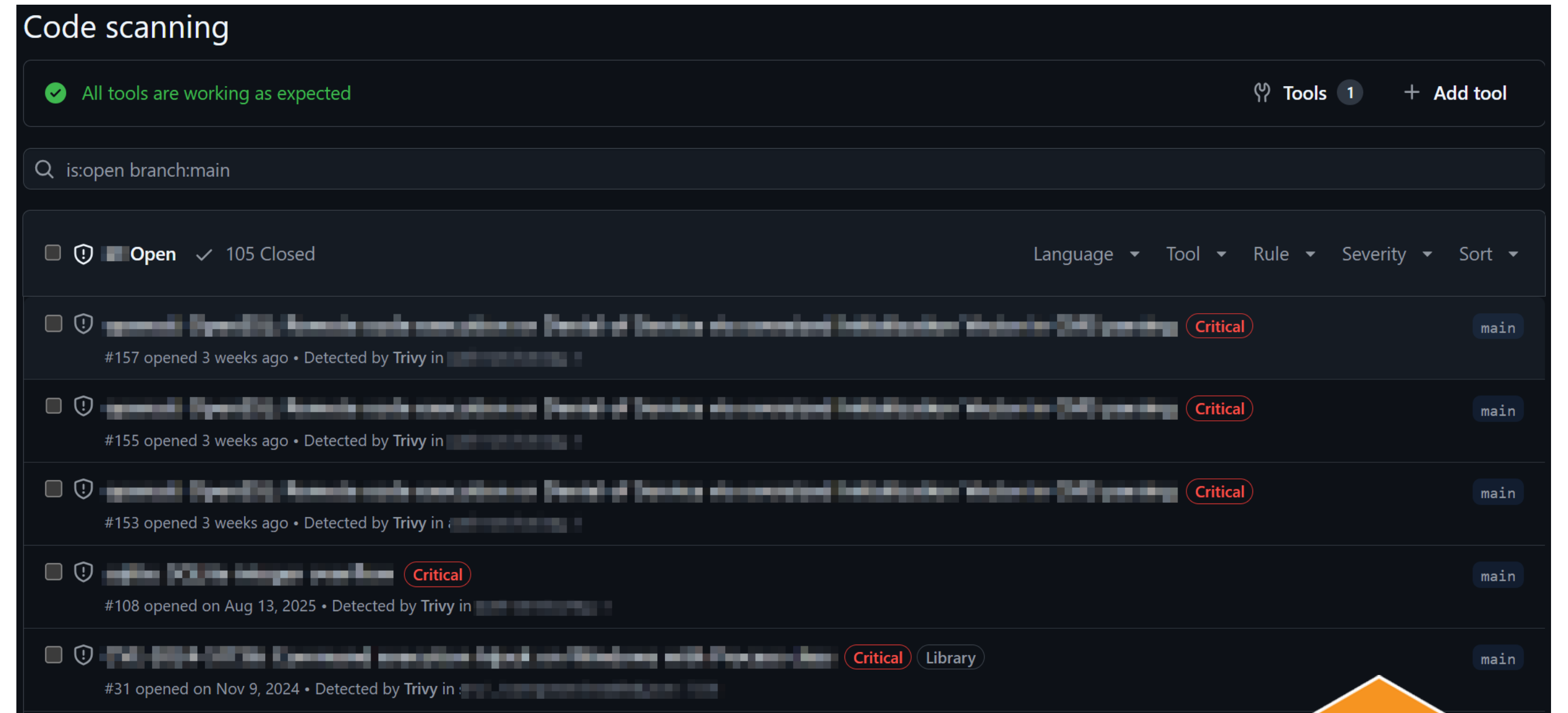
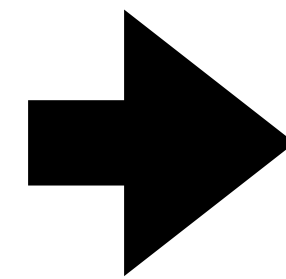


各APIイメージの最新版(latest)を取得

Trivyで脆弱性をスキャン

SARIF形式で出力

Advanced SecurityにUpload



<https://trivy.dev/>

全リポジトリの脆弱性を一元管理

重大度(Critical/High/Medium/Low)で優先度を判断

対応状況の追跡ができる

Impact analysis

Top 10 repositories and vulnerabilities that pose the biggest impact on your application security.

Repositories

Advisories

SAST vulnerabilities

Repository	Open alerts	Critical	High	Medium	Low
...	...	...	...	107	4
...	...	...	...	30	23
...	...	...	...	31	21
...	...	...	...	39	14
...	...	...	...	32	23
...	...	...	...	50	7
...	...	...	...	56	6
...	...	...	...	21	13
...	...	...	...	16	18
...	...	...	...	36	10



アラート管理のフローが未整備

- 仕組み化の検討中...

対応の属人化

- 対応フローの整備

PHPのバージョンアップなど、開発×運用の連携の難しさ

- 開発チーム、運用チームの相互理解の機会を作る
- 連携強化



おまけ

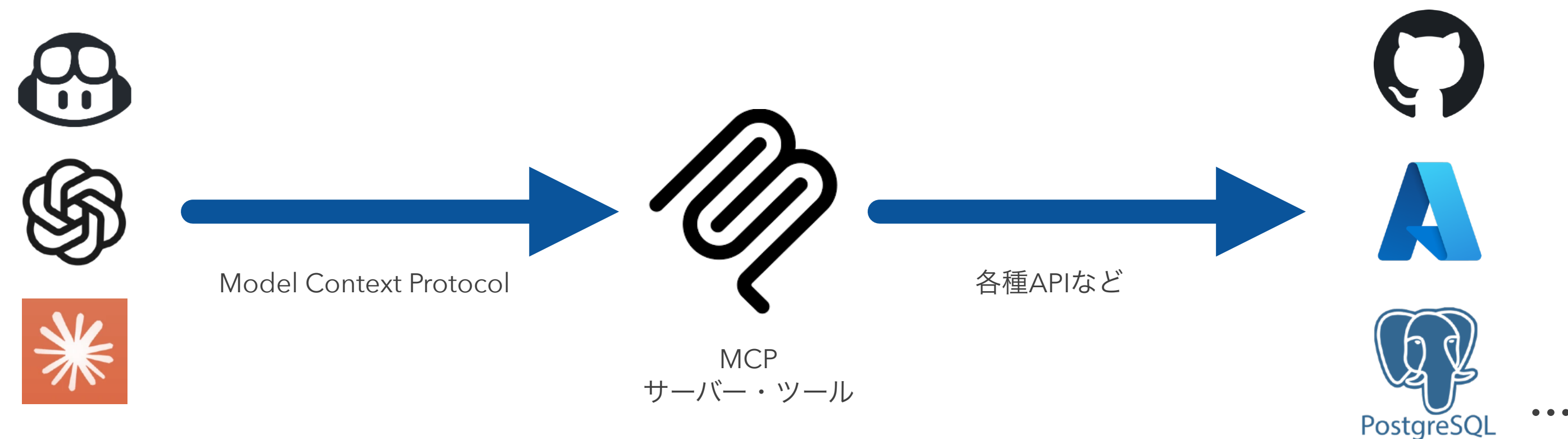
取組み中の MCP活用について

MCP (Model Context Protocol)

AIとツールをつなぐオープンなプロトコル

- ・ 2024年11月 Anthropic が公開
- ・ 2024年12月 Linux Foundation (AAILF) に移管

GitHub も公式 MCP サーバーを公開



課題

- ・ DevOpsメトリクスを可視化したい (Four Keysとか)
- ・ でも、ダッシュボード作るの面倒...

MCPなら...

- ・ UIを作らなくていい
- ・ LLMがインターフェースになる
- ・ 自然言語で問い合わせできる

ユーザー: 「my-org/my-repoリポジトリの過去1ヶ月のFour Keysメトリクスを教え

Claude: (get_four_keys_summaryツールを呼び出し)

「過去1ヶ月のFour Keysメトリクスは以下の通りです:

デプロイ頻度: 15回/月 (0.5回/日) - Performance Level: High

リードタイム: 平均 12.5時間 - Performance Level: Elite

変更失敗率: 6.7% - Performance Level: Elite

MTTR: 平均 2.3時間 - Performance Level: Elite

総合評価: Elite

デプロイ頻度をもう少し上げることで、Eliteレベルに到達できそうです。」

<https://github.com/kaz29/mcp-server-example>

<https://kaz29.hatenablog.com/entry/2026/01/12/163450>

まとめ

✓ 専門家じゃなくても、DevSecOpsはできる

GitHubの機能 + OSS を組み合わせる

完璧じゃなくていい、できるところから

✓ DevOpsの土台が大事

CI/CD、テスト自動化、メトリクス

ここがないとSecurityは載せられない

✓ 課題もある、継続的に改善

運用フローの整備

チーム間連携の強化

✓ AIの活用も進めていく

MCPでメトリクス分析を検証中

あたらしいを、あたりまえに

J:COM

